

Lazarus

The model proposes. The code decides. A revenue-recovery agent built so the LLM can never be the last line of defense on its own actions.

Repo github.com/ShاونT06/project-lazarus Runtime FastAPI “ OpenRouter

Store SQLite / Turso Batch result 50/50 resolved, 76% actioned

00 The core idea

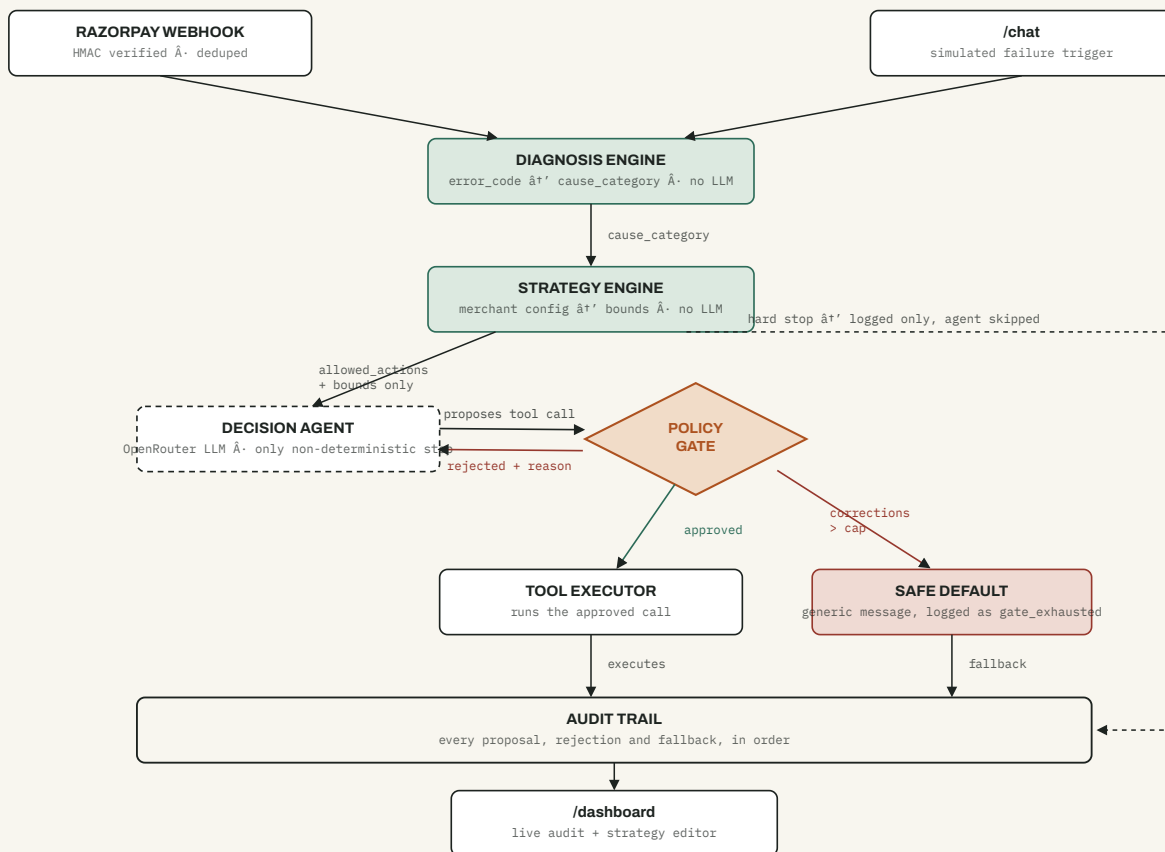
Lazarus talks to customers whose payments failed or who abandoned checkout, and tries to recover the sale “ a discount, a fresh payment link, a retry, a split-payment offer. The interesting design problem isn't the talking. It's that an LLM is choosing real actions with real consequences, and LLMs occasionally do the wrong thing confidently.

DESIGN RULE

The LLM is a **negotiator inside a fence**, never a decision-maker. It picks *which* allowed action to take and *how* to phrase it. It never decides *how much* (discount %) or *how many times* (retries) “ those bounds come from a merchant-authored strategy config, and every single tool call the model proposes is re-checked against that config in plain deterministic code before anything executes. The fence is enforced after generation, not requested in the prompt.

01 How a case moves through the system

Two deterministic stages decide *what's allowed* before the model ever runs. The model then proposes one action at a time into a gate that can reject it, correct it, or “if it keeps getting it wrong” override it with a safe default. Nothing reaches a customer without passing through that gate.



Solid borders mark deterministic code; the dashed Decision Agent box is the one non-deterministic step. Everything the model proposes passes through the gate before it can affect a real customer “approved, rejected-and-corrected, or overridden by a safe default once corrections exceed the cap.

02 The six components

DETERMINISTIC

Diagnosis Engine

Maps a Razorpay failure reason straight to a cause category â€” `insufficient_funds`, `expired_card`, `issuer_down`, and so on. No error code at all means no payment was ever attempted, so it maps to `abandoned_checkout` rather than an error. An unrecognized code falls to `unknown` instead of raising â€” a data-quality fact worth logging, not a crash. `app/diagnosis.py`

DETERMINISTIC

Strategy Engine

Evaluates the merchant's own config â€” hard stops, then rules, then defaults, first match wins â€” against the case, and resolves it to bounds: which actions are allowed, the max discount, max retries, the cooldown. The decision agent never sees the config file itself, only this resolved result. `app/strategy.py`

LLM

Decision Agent

Given the case and its bounds, picks which allowed action fits and writes the customer-facing message. Runs on OpenRouter so the model is swappable without touching the loop; the free tier stays the code default so a clean clone runs at \$0. `app/agent.py`

DETERMINISTIC

Policy Gate

Re-validates every proposed call in code, in order: hard stop active, tool in `allowed_actions`, discount under cap, cooldown respected â€” then scans the message text itself for a percentage or amount that was never approved by a tool call. That last check closes the gap where nothing would otherwise stop the model from typing â€œ50% offâ€ after a 50%-discount call was rejected. `app/policy_gate.py`

DETERMINISTIC

Tool Executor

Runs an approved call â€” mocked today (payment links, retries, CRM notes), real signatures throughout so swapping in live Razorpay calls later touches only this file, not the agent loop or the gate. `app/tools.py`

Audit Trail

Every LLM turn, every approval, every rejection with its reason, every fallback “ logged in order, keyed to the case. This is what the dashboard reads and what makes a guardrail rejection provable rather than claimed. `app/audit.py`

03 What the agent can actually call

Five tools, total. The model can only ever choose among these “ and even then, only the subset a given case's strategy rule allows.

send_message

`body: string`

Sends a recovery nudge to the customer. English only, no code-switching “ enforced by the prompt, since it's a style constraint, not a bound.

GATE CHECKS cooldown window; no unapproved % or amount in the body

generate_payment_link

`discount_pct: number`

Creates a fresh checkout link, optionally with a discount.

GATE CHECKS `discount_pct <= strategy.max_discount_pct`

generate_split_payment_link

`installments: integer`

Creates a split/installment link “ the lead remedy for overdue B2B receivables, not a plain reminder.

GATE CHECKS must be in `allowed_actions` for the matched rule

schedule_retry

`delay_hours: number`

Queues an automatic retry of the payment instead of asking the customer to act at all.

GATE CHECKS bounded by `strategy.max_retries`, enforced across the run

update_customer_record

note: string

Writes a note to the customer's CRM record â€” no customer-facing effect at all.

GATE CHECKS must be in allowed_actions â€”
see batch_sub_008 below

04 The reject-and-correct loop

One case can take several turns. The loop keeps going until the model stops calling tools, a correction cap is hit, or a hard ceiling of 8 turns is reached â€” whichever comes first.

- 1 The agent sees the case, its bounds, and a plain-English explanation of what actually happened â€” `insufficient_funds` reads nothing like `issuer_down` â€” and proposes one or more tool calls.
- 2 Each proposed call goes through the gate individually. An approved call executes immediately and its result is fed back to the model as a tool response.
- 3 A rejected call increments a correction counter and feeds the exact rejection reason back to the model, which must revise its next proposal rather than repeat the same call.
- 4 If corrections exceed the cap (default: 3), the loop stops proposing and executes one deterministic fallback message instead â€” logged as `gate_exhausted`, never silently swallowed.
- 5 A turn with *zero* tool calls, before any action has been taken, is nudged once rather than accepted as â€œnothing to doâ€ â€” a smaller model reasoning about the right call in plain text turned out to be far more common than a

genuine decision to do nothing. A no-call turn after an action has already happened is still accepted immediately as real completion.

05 The fence caught something, live

Two rejections from the final 50-case batch run “not staged, not from a test suite, from the real agent talking to real (test-mode) cases.

batch_sub_008 The model proposed `update_customer_record` for a case whose matched rule didn't allow it. The gate rejected it with the exact reason “tool not in `allowed_actions` for that rule” and the run continued correctly within bounds.

The strongest guardrail evidence in the batch: a real proposal, outside real bounds, caught by code rather than a prompt.

batch_sub_020 The model called a tool name that doesn't exist “a hallucinated function” which the gate has no way to treat as anything but rejected. The next proposal self-corrected to a real tool.

Confirms the loop's correct-and-retry path fires on a genuine model error, not just a synthetic test case.

06 Two ways in, one loop

The webhook pipeline and the customer chat UI share the exact same agent loop and the exact same gate “nothing about the chat surface gets a shortcut.

ONE-SHOT

run_case()

MULTI-TURN

The original path: a webhook or a batch script hands over a case, the loop runs to completion, and a `RunResult` comes back. Used by `scripts/run_batch.py` and the live webhook pipeline.

`start_conversation()` / `continue_conversation()`

Backs `/chat`. Messages persist across HTTP requests, so a customer's reply re-enters the same gated loop instead of starting fresh.

- A customer's message can change what the agent *says*
- It can never change what the agent is *allowed to do*
- Every proposal is re-gated on every single turn, no exceptions

07 Where it stands

Final 50-case batch, run against the live agent end to end.

50/50

CASES RESOLVED

76%

ACTIONED (38)

2

CORRECT HARD-STOP

10

NO ACTION

0

GATE-EXHAUSTED

0

ERRORS